TOKEN ECONOMICS · BUILD PLAYBOOK · 2026

Token Economics *phase* playbook

The code is Python in a monorepo. Each phase from P0 to P3 gets four things: the packages it adds, its gates, a test-driven development (TDD) format, and card titles for the GitHub Project board. Each phase also ends with a research paper. The main measure is cost per successful task (CPS), and the source of every gate is the phase plan (PHASES). It follows GitHub's own project guidance: issue forms, rulesets, required checks, Projects fields and views, sub-issues and milestones.

4	28	154	50
phases	epics	cards	fix-table rows, each with a card

Everything here comes from the phase plan and the 50-row fix table except the repository mechanics, which are DevX proposals. The code is Python 3.12 or later in a uv workspace. The first spike tests LiteLLM as the gateway core; the package boundaries hold either way.

What changed in November 2026 · Repository and GitHub setup · TDD cycle · P0 Measure · P1 Enforce · P2 Optimise · P3 Harden · Sources · Research papers

RESEARCH UPDATE · PAPERS DATED 2026-09-01 OR LATER

What changed in *November 2026*

Bottom line

No paper proves a best-possible technique for token cost. Every result in range is empirical, and some conflict. Four findings are consistent, and they change the plan.

1. **Cache reads can be the largest cost line, and token savings do not equal cost savings.** CliffCompaction's Table 7 puts cache reads at 78% of an uncompacted bill (General Language Model (GLM) 5.1 on Terminal-Bench, idealised cache model). Beyond Token Savings found Qwen running 20 to 80% slower on Terminal-Bench under token-saving policies, while Software Engineering (SWE)-bench latency stayed flat.
2. **Cheap, deterministic controls have the better evidence, with caveats.** TokenEconomizer's tool pruning, loop interruption and micro-skills cut cache-aware cost 18.4 to 30.9% on SWE-Bench, from the harness side. The harness design study found elision and summarisation about equal in accuracy, with a staged mix cheapest.
3. **A single run is not a measurement.** Run-to-run spread exceeded the gap between agents in one 312-run study (584 runs across three studies). Cost differed 22.5x between two agents on one model, from a rate card, mostly following cache share.
4. **Headline numbers shrank on full-text reading.** All 19 papers were read end to end by seven reader agents. Most savings are single-benchmark, simulated or token-based, so re-measure before quoting any of them.

This update covers papers dated 2026-09-01 or later. Every date was checked on the arXiv page (v1 submission date). The corpus was pruned at the same time: 49 papers with no cost, price or routing angle were deleted. 19 papers from the window are now in the corpus: 15 on routing, context and measurement, and 4 on latent and key-value (KV) cache compression. We then read all 19 in full text, not at abstract level, and corrected the figures below . (`research/INDEX.md`)

The 15 routing, context and measurement papers and what they change

PAPER	DATE	KEY NUMBER, WITH ITS QUALIFIER	PLAN CHANGE
AgentRouter (2609.22951)	2026-09-19	A 12M-parameter step classifier cut cost 71.8% at 97.3% of the quality, in 4.7 ms per step on an A100 graphics processing unit.	Step-level routing fits in the proxy. Keep it in "Ship second". The baseline is Generative Pre-trained Transformer (GPT)-4o, not a frontier coding model. Four templated domains, code is weakest (63.8%), no cache pricing, no confidence intervals. Replicate on our own traces.
COMPASS (2610.27822)	2026-10-20	DeepSWE 78.1% at $215, against 74.1% at $500 for the best single model.	The $215 excludes router inference and needs the 28-configuration pool (18 configurations give 63.9% at $474). Confidence intervals are wide, and on FrontierSWE no router beat the best constant. Count the router's own cost in the arm.
Pathwise-Budgeted Routing (2611.05835)	2026-11-04	Solves over 2x as many tasks only at the tightest budget ($0.025): 216.2 against 100.8 on MATH500, 85.0 against 29.8 on GPQA. At the largest budgets the gap is 2.6 and 4.6 tasks.	Model the heavy tail of response cost when you set hard per-session caps. Untested on agentic or cached workloads.
Chance-Constrained Routing (2610.22695)	2026-10-15	Sublinear cost regret with a quality guarantee that holds with high probability.	The guarantee is asymptotic and needs a known-safe fallback model. The experiments shrink the confidence radius by 0.005, so they do not run the proven algorithm, and the real-model test is 100 queries with simulated satisfaction. Template for the "third" stage only. No savings evidence.
SCRouter (2610.18566)	2026-10-13	38,869 in-domain queries (plus 2,753 out-of-distribution) over 144 model-budget actions, which is 12 models times 12 budgets. The gain is mostly on security (+32.9 points, against +3.5 on utility). Cost is tokens times a fixed price table.	Quality-only routing can pick insecure pairs. Add a security condition to the router policy. Out of distribution the gain shrinks, and the probe misclassifies about a quarter of unseen security queries.
Which Model for Which Role? (2610.26676)	2026-10-19	A substantial fraction of the Pareto frontier at about 10% of exhaustive evaluation cost, and most of it by 30%. Recall on HotpotQA is 0.58 at 10% and 0.74 at 30%.	Assign models per agent role (planner, coder, reviewer) with a sampled search, not a full grid. Evidence is offline replay on question-answering, math and SQL tasks, not coding agents.
Prompt-Adaptive Ensemble Routing (PISA) (2611.10193)	2026-11-06	Cost down about 36% in distribution and 25% out of distribution. That is 84 against 132 and 292 against 387 (units of 0.0001 dollars per query) at a hand-picked setting, for +0.60 and +0.48 points of accuracy.	A fixed single model often matches or beats adaptive ensembling (Table 5). Keep ensembles off for agent loops until a measured cost-per-success gain exists.
MultiTurnRoute (2611.10848)	2026-11-07	A 20.3% drop in simulated normalised cost against Bao on WildChat (16.7% on LMSYS), assuming an 80% cache discount and no eviction. The only billed-dollar result is 19.7% on a separate 3-family pool.	Multi-turn routing built on chat traces only. Adapt it to coding traces before quoting it.

PAPER	DATE	KEY NUMBER, WITH ITS QUALIFIER	PLAN CHANGE
TokenEconomizer (2610.22732)	2026-10-15	Cache-aware cost down 18.4 to 30.9%, Pass@1 change of -1.0 to +2.6 points, on 277 SWE-Bench tasks with 3 seeds. Three mechanisms: tool pruning, loop interruption and micro-skills. It costs more for the first six tasks or so.	Closest match to this gateway, but it runs on the harness side, not as a gateway-only black box. It learns from grader success labels through a language-model proposer plus regex detectors. Supports the context guard and loop guard. Replicate before quoting.
CliffCompaction (2609.26779)	2026-09-22	Cache reads are 78% of the uncompacted bill (Table 7: GLM 5.1 on Terminal-Bench, idealised cache model). Up to 50% cost cut, but real Kimi savings on SWE-bench are about 0%.	Drop-only is the authors' own stance, but their Table 6 shows summarisation and a sliding window close or better on some benchmarks. Keep compaction a last resort, and test drop-only against the other two on our traces.
Harness design study (2609.20804)	2026-09-17	176 settings (22 models, 4 harness variants, 2 benchmarks). Elision-only and summarisation-only reach similar accuracy, and summarisation is often cheaper (550B model at 32K on SWE-bench Verified: $1.25 against $2.57). Staged elision then summarisation wins on cost.	Do not claim elision beats summarisation. Prefer the staged mix and tag every request with harness, tool set and context strategy. Most of the study needs control of the harness.
Beyond Token Savings (2609.32961)	2026-09-26	65 policies, 3 models, 3 repeats per cell, nearly 35,000 runs. The 20 to 80% slowdown is Qwen on Terminal-Bench only. SWE-bench latency was 0.79 to 1.0x. Costs are an input-only estimate with an assumed 0.1x cache price.	Judge every compression control on billed cost and latency per solved task. This study did not bill spend either.
Total Cost of Agency (2609.23790)	2026-09-20	13.6% is injection over injection plus inference at the mid tier, uncached (11.8% of the full bill). 27.6% is depth 6. The workload is a synthetic 4-node enterprise workflow, not a coding agent.	Meter injected context per agent and per depth in the ledger. Do not quote the percentages for coding traffic.
Identical Runs, Different Results (2609.33812)	2026-09-27	584 runs is 116 + 312 + 156 across three studies. Run-to-run spread exceeded the gap between agents in Study 2 only (312 runs, one machine-learning task). The over-20x cost gap is $0.08 against $1.86 (22.5x) on one model, 3 runs each, from a rate card.	Store per-run cost distributions. Capture cache read and write tokens separately. Never publish a single-run figure. About 9x of the gap follows cache share. The cause of the cache failure is untested.
DeltaSelect (2609.19607)	2026-09-17	13 evaluations for $27.86. The 58.1% cheaper result is one run per configuration on 8 tasks. The score gain is not significant (p=0.326), and bare Luna at medium effort beat the adopted configuration on both score and cost.	Use it as a method for cheap paired A/B on a fixed task subset, not as a result. Report the p-value with the cost delta.

What the full text changed

- **Headline numbers narrowed.** Most come from one benchmark, a simulation or a token count. Only two narrow results are in billed dollars (MultiTurnRoute 19.7%, DeltaSelect $27.86 for 13 evaluations), and neither covers a coding agent in production.
- **One claim reversed.** The harness study does not show that elision beats summarisation. The two reach similar accuracy and summarisation is often cheaper. A staged mix is cheapest.
- **One claim reclassified.** TokenEconomizer is a harness-side tool with three mechanisms, not a gateway-only black box. It costs more for roughly the first six tasks.
- **Latent savings shrink end to end.** The Latent-Observation Agent (LOHA) cuts context per call 43 to 57% but whole-trajectory decoder input only about 8%, and plain masking matches its resolve rate on Qwen3. Every latent method needs open weights.
- **Cost-of-agency and variance figures are narrow.** 13.6% and 27.6% are from a synthetic enterprise workflow. The variance-over-gap result is one study of 312 runs on one task.

Latent compression: agents that skip reading tool output as text

Four papers let the agent take tool output as latent (soft) tokens or compressed key-value (KV) entries instead of text. All four need access to model internals. They work only on self-hosted open-weight models. A hosted API such as Claude or OpenAI's models accepts text and cannot take injected embeddings.

PAPER	DATE	KEY NUMBER, WITH ITS QUALIFIER	LIMIT
Coca-Cola, "Tools Need Not Speak in Text" (2611.06500)	2026-11-04	Tool observations compress into latent tokens injected into the agent's input embeddings. Tokens fell 70.01 to 89.24% and runtime fell 24.45 to 55.97%. Accuracy rose only against the vanilla baseline, with no variance or evaluation sizes.	Self-host only: needs embeddings and gradients, and one compressor trained per agent. Tested on 4 web-search question-answering benchmarks with Qwen3.5 and GLM, not coding.
Agentic Soft Token Compression (ASTC) (2610.41291)	2026-10-30	50.68% fewer attention pairs is the best case (OfficeBench, -1.05 points of accuracy). AppWorld gives 19.30% fewer pairs, and its accuracy gain is about 2 tasks.	"Training-free" still needs a calibrated reader per model and tuned thresholds, plus a hybrid GDN-attention model. Pairs are analytical compute counts, not billed cost.
LOHA with Anchored Context Distillation (ACD) (2609.31430)	2026-09-25	Old observations become soft tokens and the last K stay text. K=3 cuts context per call 43% and 57%. Whole-trajectory decoder input falls only about 8% on Qwen3 (1.65M to 1.51M tokens), because calls rise from 37.9 to 60.9.	Resolve rate fell: 12.1 and 21.8% against 14.5 and 27.5% for the uncompressed bases. Plain masking matches it on Qwen3 (12.2%). The 1.9x throughput is against its own weakened agent, about 1.09x against the base.
ActKV (2609.31395)	2026-09-25	98.53% accuracy at 25.98% of peak KV memory is the mean of ALFWorld and WebShop with the adaptive budget (HotpotQA-Web: 96.81% at 35.08%). Token throughput 3.97x, task throughput 3.58x.	Needs custom vLLM kernels, so self-host only. Measured on one GPU, offline, on two open models. Not tested on coding. Absolute accuracies and intervals are not stated.

What it means here:

1. **API backends: no change.** The gateway cannot inject latents. Text-level elision (TokenEconomizer, CliffCompaction) stays the default.
2. **Self-hosted backends: a P3 candidate.** If we serve open models, a latent compressor in front of web-search tools is the highest-saving item found. Test it on our own traces first.
3. **Judge on cost, not tokens.** Every figure above is tokens, compute or memory. Beyond Token Savings (2609.32961) shows that token cuts can still raise bills.

Earlier context from before the date window: CoACT (2607.02911), TokenPilot (2606.17016), What Does Context Compression Cost an Agent? (2608.16370) and LatentPress (2609.01507, 2026-09-01). They agree with the plan. A compression control must keep the prompt prefix stable, and its cost must include re-fetching dropped state.

Price and billing changes to load into the price book

We read rows marked "Vendor page" on the vendor's own page on 2026-11-10. The other rows come from third-party reports. Confirm those against the vendor page before they go live. The price book already requires dated, expiring entries (`04-cost-formula.md`).

CHANGE	EFFECTIVE	SOURCE
Anthropic cache-read multiplier is set per model: 0.025x on Fable 5.1 and 5.2, 0.05x on Opus 5.5 and Sonnet 5.5, 0.1x elsewhere. Sonnet 5.5 reads at $0.10 per M tokens and Opus 5.5 at $0.20.	2026-10-07 for Sonnet 5.5	Vendor page; MIXED News for the date
Haiku 5.5 prices each request by length. Above 100K tokens, input rises from $0.10 to $0.50 and output from $0.50 to $2.50, even on a cache hit.	2026-10-07	Vendor page
Sonnet 5 stays at $2 input and $10 output. The planned rise to $3 and $15 on 2026-09-01 was cancelled.	2026-09-01	Vendor page

CHANGE	EFFECTIVE	SOURCE
Claude 4.7 and later produce about 30% more tokens for the same text. `inference_geo: "us"` adds 1.1x to every token category.	Current	Vendor page
OpenAI adds a cache-write charge on Generative Pre-trained Transformer (GPT) 6 and 6.1 models, and an optional 12-hour cache. Sources conflict on the write multiplier: 1.25x of input on a third-party price sheet (Sol: $2 input, $2.50 write), 2.5x in an OpenAI changelog snippet.	2026-10-20	Conflict. Read OpenAI's own page before use.
Gemini 4 Argon: input $2 to $4, output $10 to $20, cache $0.10 to $0.20 per M tokens.	2026-12-01	Vendor page
Gemini 3.8 Flash and 3.6 Flash rates double (input $0.75 to $1.50, output $3.75 to $7.50). Cache storage rises from $0.50 to $1.00 per hour.	2027-01-01	Vendor page; Yahoo Finance
Grok 4.7 doubles every rate for the whole request at 200K prompt tokens and above (input $2 to $4, output $6 to $12).	Current	Vendor page

Three rules follow.

- Give every price entry an effective date, so a historical cost-per-success trend can be restated across a cutover.
- Treat prompt length as a price input. Haiku 5.5 has a cliff at 100K tokens, and Grok 4.7 has one at 200K.
- Reconcile client-side cost displays against the provider's usage and cost API. They can lag the real price: Claude Code 2.1.295 priced Sonnet 5.5 cache reads at $0.20 until 2.1.296.

Gateway, router and agent CLI releases since September

No new research paper describes an orchestrator. The changes are in products. Dates come from vendor pages and changelogs, which we read on 2026-11-10.

PRODUCT	DATE	WHAT CHANGED	WHAT IT MEANS HERE
LiteLLM Auto Router benchmark	2026-09-11	25 Software Engineering (SWE) tasks from SWE-bench Verified, one attempt each. The router solved 23 for $11.15 ($0.48 per solved task). Always-Opus 5 solved 23 for $20.27.	Cost fell 45% at the same score. The sample is tiny and vendor-run. Replicate before you quote it.
LiteLLM Switchyard router	2026-11-06	Routes each turn by tool history (failures, retries, recovery), not prompt text. No cost numbers published.	Tool-trajectory signals are a cheap routing input for agent traffic.
LiteLLM cost accounting redesign	2026-10-31	Design post: usage meter, cost compiler, budget enforcer. Reservations are separate from settled spend.	Matches the reserve-then-settle flow in 05-policy-router-budgets.md. This is a design, not a shipped feature.
LiteLLM v1.107.0	2026-10-21	A report says the release was compromised. Releases later moved to Tuesdays, with an LTS line on 1.108.	Pin gateway dependencies. Do not use v1.107.0. We did not read the details.
OpenRouter routing rebuild	2026-10-28	The default now scores cost, speed and reliability, not cheapest-first.	Weigh latency and error cost next to price. The figures came from a search snippet.
OpenRouter Auto Code router	2026-11-06	Picks the model and reasoning effort for each request.	Effort is a routing dimension.
Claude Code 2.1.316 to 2.1.319	2026-10-31 to 2026-11-06	Pooled group budgets, a per-developer cost report API, `/usage` spend behind a gateway, a subagent effort variable.	The native gateway now enforces budgets. Integrate with it.
Codex CLI 0.166.0	2026-10-29	Rollout budgets can be advisory, with optional enforcement.	Expose both advisory and enforced modes.

Which model for which job

No peer-reviewed result gives a rule. Two leaderboards and Anthropic's own guidance agree on the shape. Treat the numbers as leads, not facts: one leaderboard page has no date, and the other is a secondary source.

- Anthropic's models page tiers the family: Haiku 5.5 for classification and routing, Sonnet 5.5 for most work, Opus 5.5 as the default for most workloads, Fable 5.2 for long-horizon agent work.
- On the Snorkel SWE-bench CLI board (undated page), cost per task spans $1.44 (GPT-6.1 Sol, 9.8% pass at 1) to $118.64 (Opus 5, 14.0%). Opus 5.5 scores 16.6% at $15.87. Dividing cost by pass rate gives about $96 per solved task for Opus 5.5 and about $15 for GPT-6.1 Sol.
- On Terminal-Bench 4.0 (Morph's page, updated 2026-10-06), Opus 5.5 scored 64.8% at $14.30 per task and Sonnet 5.5 scored 61.8% at $22.22. The mid-priced model cost more.
- Effort changes both score and cost, so route on model and effort together.

Plan files that change

FILE	CHANGE
`04-cost-formula.md`	Add effective-dated price rows and the 12-hour cache write multiplier. Add a break-even rule per prefix: the reuses needed before a 2.5x write pays off.
`05-policy-router-budgets.md`	Add per-step routing evidence (AgentRouter) with its qualifiers, the heavy-tail budget model, a security condition on routing, per-role Pareto assignment, and the COMPASS pool-size and router-cost caveats.
`06-context-guard.md`	Keep compaction a last resort and test drop-only against summarisation and a sliding window. Use a staged elision-then-summarisation mix, not elision alone. Add latent compression as a self-hosted-only candidate, and note TokenEconomizer is harness-side.
`07-observability-eval-rollout.md`	Require per-run cost distributions, a paired bootstrap with multiple-comparison correction, a metered-calls-per-task field, and the single-study scope of the variance result. Add a rule that no compression control ships on token count alone.
`09-risks-and-open-questions.md`	Record that no proven best-possible technique exists, that several vendor numbers rest on third-party pages, and that September to November figures were narrowed by full-text reading.
`14-research-papers.md`	Cite the new papers in the matching paper outlines, with the qualifiers found in full text.

What this search did not find

- No paper dated September or later on thinking-budget or reasoning-effort selection.
- No official vendor announcement of a new cost-cutting orchestrator. Bifrost and Context Gateway show no new cost feature in the window.
- No formal bound for prompt-cache eviction or agent context management. The only formal result found, a Θ(n) competitive ratio for online KV-cache scheduling (2608.16944), is dated 2026-08-14.
- No paper on billing reconciliation or on cost-of-pass as a formal metric.

Excluded on date: An Empirical Cost Attribution of Context-Compression Gateways (2609.22114) has a September ID but was submitted 2026-08-19. Token Reduction Is Not Cost Reduction (2607.12161) was submitted in July, and Prompt-Induced Waste in Coding Agents (2608.01347) in August. All three stay in the corpus as earlier evidence that the plan already cites.

REPOSITORY AND GITHUB SETUP

One repo, one board, *one* required check

This is the end-state tree: a Python monorepo of 45 packages. Each phase section below lists the packages it adds. The existing `plan/` and `research/` folders stay where they are. Full package map: `plan/13-monorepo-layout.md`.

```
token-economics/
├─ apps/         gateway  admin-api  worker  cli                          deployables, compose everything
├─ services/     ledger  pricebook  signals  health  reconciler  policy
│                insight  eval  rollout  tenancy                          stateful, own a store, serve a port
├─ features/     budgets  turn-cap  loop-guard  sticky-model  rules-router  cascade
│                cache-planner  schema-deferral  observation-masking ...  one capability each, behind a flag
│                upstream  harness-adapters  headless-runner               provider and harness files live inside
├─ modules/      contracts  metering  pricebook-core  stats  redaction
│                joiner                                                     pure libraries
├─ papers/       T1 ... T14                                               one folder per paper: draft, data, code, figur
├─ evals/        task sets, bounds, results
├─ ops/          alerts, load tests, drills
├─ spikes/       throwaway, never imported
├─ tools/        layering, extractability and surface checks, release
├─ docs/         adr/  runbooks/  evals/
├─ plan/  research/  research-cli-agents/    existing, unchanged
├─ .github/      CODEOWNERS, ISSUE_TEMPLATE/, PULL_REQUEST_TEMPLATE.md, workflows/ (ci-gate), dependabot.yml
├─ pyproject.toml  uv.lock  .python-version  ruff.toml  .importlinter
└─ README.md  CONTRIBUTING.md  SECURITY.md  CHANGELOG.md  LICENSE  .gitignore
```

Monorepo rules

RULE	HOW IT IS ENFORCED
Five layers, one direction: contracts, modules, features, services, apps. A package imports only from layers to its left.	import-linter contract generated from layout.json; required check layering
No sideways imports. Features, harness adapters and backends meet through entry points and the ports in contracts.	import-linter forbids sibling imports; contract test suite per port
Contracts is the only shared code: types, ports and events. No I/O.	Dependency list of the contracts package is checked in CI
Each folder ships alone: own pyproject, tests, intro file (README), changelog and version.	Extraction test builds each package in a clean venv with only its declared deps
Per-package tags and releases, for example feat-budgets/v0.3.0. Changes to contracts bump its major version when breaking.	release-please in manifest mode (DevX proposal); dependents tested against both ends of the contracts range
CI runs only affected packages and their dependents.	Path filter computed from layout.json, aggregated by ci-gate
A folder can leave the repo with its history.	git subtree split on the package path

---

Research papers

Each phase ends with a paper that reports what it measured. A paper never blocks a phase gate. Titles, data needs and citations: `plan/14-research-papers.md`.

ID	SLOT	WORKING TITLE	PHASE GATE THAT PRODUCES IT
T1	P0, week 4	Cost per Successful Task, Specified: Numerator, Denominator and Failure Handling for Billed Coding-Agent Spend	P0 exit: baseline exists
T2	P0, week 4	Five Exclusive Token Buckets: Auditing Usage Schemas Across Five Vendors	P0 exit: ledger matches invoice
T3	P1, week 6	Where the Billed Dollar Goes: A Four-Component Cost Decomposition of Three Production Coding Harnesses	P0 data, written during P1

ID	SLOT	WORKING TITLE	PHASE GATE THAT PRODUCES IT
T4	P1, week 8	How Much Does a Soft Budget Leak? Overshoot of Post-Hoc vs Reserve-and-True-Up Caps	P1 exit: budgets enforced
T5	P1, week 8	Telling the Agent vs Stopping the Agent: Visible Budget Reminders vs Gateway Caps	P1 exit
T6	P2, week 10	Paired, Task-Clustered Evaluation of Cost Optimisations with Cache Isolation and Non-Inferiority Margins	P2 week 2: harness runs
T7	P2, week 14	5 Minutes or 1 Hour? Planning Prompt-Cache Lifetime from Measured Reuse-Gap Distributions	P2 exit
T8	P2, week 14	Does a Router Beat the Best Single Model After Overhead and Cache Loss?	P2 exit
T9	P2, week 14	Same Compressor, Same Model, Different Harness: Why Output Compression Moves Billed Cost in Opposite Directions	P2 exit (replication only)
T10	P3, week 16	When Does Another Attempt Stop Paying? Dollar-Denominated Stopping Rules for Agent Retries and Step Caps	P3 week 2
T11	P3, week 18	Bounding the Bill, Not Detecting the Prompt: Strong-Model Share Ceilings as a Router Defence	P3 week 4
T12	P3, week 18	What a Gateway Can See That a Transcript Cannot, and the Reverse: Waste Detection Across Two Vantage Points	P3 exit
T13	P3, week 18	Price Books Go Stale: Lifecycle and Canaries for Billing-Accurate Cost Tracking	P3 exit
T14	After P3	Token Economics: A Harness-Agnostic Cost Control Plane for API and CLI Coding Agents	Release 1.0

Venues are not chosen. The novelty check marks 4 titles open, 9 partly covered and 1 mostly covered; partly covered titles claim only the uncovered part.

Paper execution plan

Detail per paper, with claim, pre-registration, power, baselines, reproducibility and referee objections: `plan/15-paper-execution-plan.md`. Research-method skills wrote it, and an independent reader checked it. **No title is likely to reach a Nature-family venue.** The series targets top machine-learning (ML) and systems venues. Every venue name below is a proposal to verify at submission.

Terms: Mining Software Repositories (MSR), International Conference on Software Engineering (ICSE), Transactions on Machine Learning Research (TMLR), design effect (DEFF), intraclass correlation (ICC), non-inferiority (NI), time to live (TTL), Open Science Framework (OSF).

ID	NATURE-FAMILY PLAUSIBILITY	WHY	REALISTIC VENUE
T1	Remote	A metric definition. Two 2026 papers already use the metric.	MSR or an empirical-software-engineering journal
T2	Remote	An audit. Useful, narrow.	MSR data-showcase track, or workshop
T3	Remote	Three harnesses, small user base.	MSR, ICSE software-engineering-in-practice track
T4	Remote	Engineering measurement.	MLSys or a systems workshop
T5	Low	Behavioural question with a clear conflict in prior work. Needs many tasks.	NeurIPS main or workshop, ACL-family if framed on language agents
T6	Low, the best case	A method with wide reuse. Would need a large multi-lab validation.	NeurIPS Datasets and Benchmarks, or TMLR

ID	NATURE-FAMILY PLAUSIBILITY	WHY	REALISTIC VENUE
T7	Remote	A planning rule tied to vendor price multipliers that can change.	MLSys, or a systems venue
T8	Low	Strong question. Needs real traffic from more than one organisation.	NeurIPS Datasets and Benchmarks, or EMNLP industry track
T9	Remote	Mostly covered. Replication value only.	ReScience-style venue or workshop
T10	Low	A general stopping-rule result could travel, if the knee holds across models.	ICML or NeurIPS main, or TMLR
T11	Remote	The bound holds by construction (section 3, T11).	Security workshop, or fold into T8
T12	Remote	Method comparison on one system.	ICSE or FSE tools track
T13	Remote	Thin evidence. Fold into T2.	Section of T2
T14	Remote	A system paper from one team.	EuroSys or USENIX ATC, then MLSys; OSDI only if the evidence is much larger

Claim, statistics and calendar per paper

ID	CLAIM AND NULL	STATISTICS AND SAMPLE SIZE	CALENDAR AND STOP RULE
T1	Counting failed attempts, retries, router and judge cost raises CPS by more than 5% over success-only cost, or changes the harness ranking, on the P0 30-day ledger. Null: the rise is 5% or less and the ranking is unchanged.	Unit is the task. Statistic is the log ratio. Cluster bootstrap CI by repository. Holm across at most 3 harnesses. Sizing is by precision. A CI half-width of 0.05 on the log ratio with SD 0.312 needs `(1.96*0.312/0.05)^2 = 149` tasks, 283 with DEFF 1.9. This is a function of assumed `tau`. P0 baseline measures it.	Calendar week W4 in `14-research-papers.md`. Plan: register W1, freeze W4, preprint W6, submit W8. Gate W7. Phase gate P0 exit (baseline exists). Do not submit alone if the CI on the log ratio includes 0.05 and the ranking holds. Fold into T14.
T2	Naively summing reported input fields mis-states billed tokens by more than 2% on at least one of five vendors. The 2% tolerance is the plan's attribution figure (unverified). Null: error is 2% or less on all five.	Unit is the response id. Clopper-Pearson CI on mismatch share. Holm across 5 vendors. Zero-failure sizing. To show a mismatch rate below 1% with none seen needs 299 responses per vendor. Below 0.1% needs 2,995. Effect size is the relative dollar error.	W4 in the plan. Register W1, freeze W4, preprint W6, submit W8, gate W7. Phase gate P0 exit (ledger matches invoice within 1%). Do not submit if no vendor exceeds the 2% tolerance. Publish the suite as a package note instead.
T3	The share of billed dollars from tool output plus history differs across Claude Code, Codex and one other harness. Null: the share vectors are equal across harnesses.	Unit is the session, clustered by developer. Isometric log-ratio transform, then permutation test on the harness label. Cluster bootstrap CI. Holm across 3 harness pairs. Sizing. A share CI half-width of 0.05 at p = 0.5 needs 385 sessions, 730 with DEFF 1.9, per harness. Assumed ICC.	W6 in the plan. Register W2, freeze W5, preprint W8, submit W10, gate W9. Written in P1 from P0 data. Do not submit if the share vectors differ by less than the bootstrap CI. Fold into T14.
T4	The 95th percentile overshoot under post-hoc caps is more than 2 times that of reserve-then-true-up caps on the same sessions. The factor 2 is a decision value. Null: ratio is 1 or less.	Unit is the session. Primary endpoint is the P95 overshoot ratio, tested by a quantile bootstrap CI. Paired Wilcoxon signed-rank on median overshoot is secondary and reported without a claim. One primary, so no correction. Sizing. P95 needs about 20 exceedances to be stable, so 400 sessions. Assumed. Flag.	W8 in the plan. Register W5, freeze W8, preprint W10, submit W12, gate W11. Phase gate P1 exit. Do not submit if the ratio CI includes 1.
T5	A visible budget reminder changes spend per task, and a gateway cap changes it more. Null: spend equals the no-intervention arm. Prior work conflicts (C:131 helps, Ramp Labs reports no change).	Unit is the task. Paired log difference. Cluster bootstrap CI. Holm across 2 contrasts. NI on success at 10 pp. Sizing. 10% cost effect needs 145 tasks per contrast (section 3.2). NI needs 221 (section 3.3). Both assumed.	W8 in the plan. Register W5, freeze W8, preprint W10, submit W12, gate W11. Phase gate P1 exit. Do not submit if both CIs include zero and the paper adds no insight beyond the null. Post as a registered null.

ID	CLAIM AND NULL	STATISTICS AND SAMPLE SIZE	CALENDAR AND STOP RULE
T6	Without per-arm cache isolation, a no-op change shows a spurious cost difference larger than the isolated-arm CI half-width in at least one A/A run. Null: no spurious difference.	Unit is the task. Rejection rate with exact binomial CI. Zero-failure sizing. To show a false-positive rate below 10% with no rejections needs 29 A/A runs. To show below 5% needs 59. Effect size is bias as share of the true difference. Single primary, no correction.	W10 in the plan. Register W9, freeze W10, preprint W12, submit W14, gate W13. Phase gate P2 harness runs. Do not submit if the A/A false-positive rate is within the nominal 5% without isolation.
T7	A planner that picks the TTL from the measured reuse gap lowers cache cost per 1,000 requests versus a fixed 5-minute TTL. Null: no difference.	Unit is the caller. Sign-flip randomisation test on paired cost differences. CI by bootstrap over callers. No correction for one primary. Sizing. With n callers the smallest two-sided p is `2/2^n`, so at least 6 callers are needed. Effect size depends on the gap distribution, so the P0 baseline measures it. Not computable now.	W14 in the plan. Register W9, freeze W13, preprint W16, submit W18, gate W17. Phase gate P2 exit. Do not submit if fewer than 6 callers or if the planner never beats the fixed policy.
T8	After overhead, judge cost and cache loss, at least one router lowers CPS versus the best single model with its own cache, with success within 10 pp. Null: every router has CPS at or above best-single.	Unit is the task. Paired log CPS. Cluster bootstrap. Holm across 4 routers. Intersection-union with NI. Sizing. 221 tasks for NI at 10 pp, `pD` 0.2. 145 for a 10% cost cut. Assumed.	W14 in the plan. Register W9, freeze W14, preprint W16, submit W18, gate W17. Phase gate P2 exit, router beats best single or stays off. Do not submit as a win if no router passes. Post as a registered null.
T9	One output compressor shifts billed cost in opposite directions on two harnesses. Null: the log effect has the same sign and size on all harnesses.	Unit is the task. Interaction on the log scale. Cluster bootstrap. Holm over mechanism metrics. Sizing. At interaction 0.20 about 39 independent tasks per harness, 20 if paired. At 0.528 (the PointFive gap) 6 and 3. At 0.10 it is 153 and 77. Assumed.	W14 in the plan. Register W9, freeze W14, preprint W16, submit W18, gate W17. Phase gate P2 exit. Do not submit if the replication shows the same sign everywhere. A null replication still goes on a preprint.
T10	A dollar-denominated stopping rule lowers CPS versus the fixed step cap at P75 of baseline, with success within 10 pp. Null: no CPS difference.	Unit is the task. Paired log CPS. Cluster bootstrap. Holm across 2 baselines. Sizing. 145 tasks for a 10% cost cut. Carryover bias of 10% needs 69 tasks. Replay needs attempts to be independent, which the carryover check tests.	W16 in the plan. Register W12, freeze W16, preprint W18, submit W20, gate W19. Phase gate P3 week 2. Do not submit if the knee CI spans 1 to 5 attempts.
T11	With a strong-share ceiling `s` and price ratio `rho`, cost per request cannot exceed `s*rho + (1-s)` in weak-price units, and success on benign traffic stays within 10 pp. Null: a request exceeds the bound or success drops more.	Unit is the request. Exact binomial bound. To bound the violation rate below 1% with none seen needs 299 requests. Below 0.1% needs 2,995. Benign NI as T8. The bound holds by construction, so H1 tests the code and not the idea.	W18 in the plan. Register W14, freeze W17, preprint W20, submit W22, gate W21. Phase gate P3 week 4. Default decision is to fold into T8 unless the benign-success cost is new.
T12	At least one waste pattern class is detectable only at the gateway and at least one only in transcripts. Null: the two vantage points detect the same classes with equal recall.	Unit is the session. McNemar per pattern. Benjamini-Hochberg over the 50 rows. Sizing. For a 20 pp recall gap with discordance 0.3, two-sided 0.05, 80% power, 59 reference-positive events per pattern. Rare patterns may fall short and are reported descriptively.	W18 in the plan. Register W12, freeze W18, preprint W20, submit W22, gate W21. Phase gate P3 exit. Do not submit if kappa is under 0.7 after adjudication.
T13	After a vendor price change, tracked cost drifts more than 1% from the invoice within 7 days without a canary, and a weekly canary catches it. The 1% is the P0 gate. Null: drift stays within 1%.	Unit is the price-change event. Exact binomial CI. To show a miss rate below 10% with no misses needs 29 events. Real events are unlikely to reach that, so injected events carry the number. Not computable until the P0 price-row log counts real changes.	W18 in the plan. Fold decision W10. Otherwise register W12, freeze W18, preprint W20, submit W22, gate W21. Phase gate P3 exit. Do not submit standalone if real events are under 5.
T14	The composed Token Economics system lowers CPS versus the same harness without Token Economics on a frozen task set, with success within 10 pp. Null: no difference.	Unit is the task. Paired log CPS. Cluster bootstrap. Intersection-union across the two endpoints, so both must pass; no Holm needed. Sizing. 221 tasks for the NI test at 10 pp, 145 for a 10% cut. Assumed.	About W22 in the plan, an estimate. Register W15, freeze W22, preprint W24, submit W26, gate W25. Phase gate Release 1.0 after P3 exit. Do not submit if the composed A/B fails NI or shows no cost gain. Publish the preprint as a report.

Pre-submission gate

A paper submits only if every row passes. A second person checks, matching the phase rule that someone other than the builder verifies.

#	GATE	PASS WHEN	EVIDENCE
1	Registered	OSF record dated before confirmatory data	Registration id, hash list
2	Frozen artifacts	Task manifest, analysis tag, price-book snapshot, model ids recorded	Hash file in `papers/<id>/`
3	Endpoints intact	Every registered endpoint is reported, including failures	Endpoint table
4	Dry run	A no-op change reports no difference	Dry-run log
5	Reproduction	Folder runs from a clean checkout	CI run id
6	Numbers traced	Each number has a script that regenerates it	Script list
7	Tenant-data check	No prompts or tenant ids in released data	Storage scan report
8	Prior-art refresh	Novelty check re-run at draft start and at submission	Dated note
9	Overlap scan	Sibling-paper overlap disclosed and similarity checked	Overlap table
10	Statistics review	Independent person reruns the analysis	Review note
11	Declarations	Conflicts, funding, AI use, data and code statements, CRediT	Declarations page
12	Venue rules	Page limits, policy on preprints and AI use checked	Dated policy record (verify at submission)
13	Go/no-go	The paper's decision rule in section 5 row 11 is met	Signed gate note
14	Independent review	A reader outside the authors scores the draft against the referee table	Review notes

Sample sizes assume effect sizes that a named pilot will replace. Venue policies, the Nature reporting-summary fields and several vendor figures are unverified in the plan.

GitHub settings

AREA	SETTING
Branches	Trunk-based. Short branches named `type/issue-slug`, for example `feat/42-ledger-dedupe`. Squash merge only. Delete branch on merge.
Ruleset on main	Pull request required. One approval from someone other than the author. Code owners review. Conversations resolved. No force push. Linear history. One required status check: `ci-gate`.
ci-gate job	A final job that depends on every other job and fails if any failed or was skipped by mistake. Only this check is required, so path filters never leave a check pending.
Commits and PR titles	Conventional Commits: `feat`, `fix`, `test`, `docs`, `chore`, `spike`, `gate`. The PR title becomes the squash commit. Scope is the folder name.
Issue forms	Five issue forms in YAML (a plain-text configuration format): epic, story, spike, fix-row, gate. Each has required fields: done-when line, test name, size, dependency.
PR template	Linked issue; link to the red test run; evidence for the done-when line; flag state; rollback note.
CODEOWNERS	One owner per package folder. Security, pricing and budget packages need two owners.
Labels	`phase:P0` to `phase:P3`; `type:epic|story|spike|fix-row|gate|bug|docs`; `area:gateway|adapters|ledger|pricing|detectors|budget|policy|eval|router|rollout|security|ops|paper`; `class:0|1|2|3` for fix-table rows.
Milestones	One per phase, due on the end date in PHASES.md. The gate card closes last and closes the milestone.

AREA	SETTING
Releases	Tag each phase on the repository: v0.1.0, v0.2.0, v0.3.0, v1.0.0. Each package also has its own tags and changelog. Generate changelogs from commit titles.
Decisions	One architecture decision record (ADR) per choice that is costly to reverse, kept in docs/adr/. Status: proposed, accepted, superseded.
Security	Secret scanning and push protection on. Dependabot on. SECURITY.md names the reporting address.

Project board

One GitHub Project for the repository. Epics are issues with sub-issues. Cards are issues, so every card has a number, a branch and a PR.

COLUMN	ENTRY RULE	EXIT RULE
Backlog	Card exists with a title in the convention	Passes Definition of Ready
Ready	Has a done-when line, a named test, a size of 2 days or less, and linked dependencies	Someone picks it up
In progress	Branch open. At most 2 cards per person.	Red test committed, then green
In review	PR open, ci-gate green, evidence attached	One reviewer approves
Verify	A person other than the builder re-runs the evidence	Evidence reproduced
Done	Merged and flag state recorded	Closed by the PR

PROJECT FIELD	VALUES
Status	The six columns above
Phase	P0, P1, P2, P3
Epic	Parent issue
Row	Fix-table row number 1 to 50, or empty
Class	0 alert, 1 guard, 2 behaviour change, 3 human action
Size	0.5, 1, 2 days. Anything larger is split.
Owner	One person

VIEW	LAYOUT	USE
Board	Board, grouped by Status, filtered to the open phase	Daily stand-up
Roadmap	Roadmap, by milestone	Phase dates and gate dates
Rows	Table, filtered to type:fix-row, grouped by Class	Track the 50 rows to a result file
Gate	Table, filtered to type:gate	See which exit gates are open

Definition of Ready and Done

Ready: a card may start

- The card has a title in the Conventional Commits form.
- It copies one done-when line from PHASES.md.

Done: a card may close

- A failing test was committed before the code, and the PR links that run.

- It names the test that will prove it.
- It takes 2 days or less.
- Its dependencies are linked and not blocked.

- ci-gate is green.
- The evidence named in the done-when line is attached.
- A person other than the builder reproduced it (the Verify column).
- Docs and the ADR are updated if a decision changed.
- New flags default to off.

Card title convention

A title reads like the commit that will close it, so the board and the changelog match.

```
type(scope): imperative summary

feat(ledger): one row per provider response id
fix(row-11): stale tool results carried forward, paired CPS test and result file
gate(P1): exit review, key cannot exceed budget beyond the measured overshoot
```

Types: feat, fix, test, docs, chore, spike, gate. A fix-row card carries the row number from the fix table so the board can show all 50.

TDD CYCLE

Red, green, refactor, then *evidence*

1. **Red.** Commit a failing test named after the card's done-when line. The PR links the failing run.
2. **Green.** Commit the smallest change that passes.
3. **Refactor.** Clean up with the tests green. Skip if there is nothing to clean.
4. **Evidence.** Attach the artifact the done-when line names: a replay diff, a fault report, a result file.
5. **Verify.** A second person reproduces the evidence from the PR before the card moves to Done.

Tests follow Given, When, Then in comments, and a name that states the behaviour. Each phase adds a layer.

TEST LAYER	PROVES	INTRODUCED
Unit and property	Maths, normalisation, parsing; invariants such as the five buckets summing to the total	P0
Contract	Every adapter honours route, identify, observe, cap and replay the same way	P0
Replay	Recorded traces behave the same through the gateway, byte for byte or with a named delta	P0
Fault and concurrency	No lost or doubled ledger rows; budgets hold under parallel load	P1
Paired eval	A fix lowers CPS while success stays inside the declared bound	P2
Chaos and drill	Rollback, kill-switch and recovery work without a deploy	P3
Load	The gateway holds its latency budget	P3

P0 · 4 WEEKS · MILESTONE P0 MEASURE, TAG V0.1.0

P0: *measure only*

Watch and price every request. Nothing is blocked or changed.

7	49	9	4
epics	cards	PR and nightly checks	exit-gate lines

Folder structure

Packages this phase adds, plus the shared folders it touches. Every package has its own pyproject, tests, intro file (README) and changelog. Layering rules are in the repository section.

```
token-economics/
├─ modules/  (1 new)
│  └─ contracts
├─ modules/  (4 new)
│  ├─ joiner
│  ├─ metering
│  ├─ pricebook-core
│  └─ redaction
├─ features/  (2 new)
│  ├─ harness-adapters
│  └─ upstream
├─ services/  (3 new)
│  ├─ ledger
│  ├─ pricebook
│  └─ signals
├─ apps/  (2 new)
│  ├─ cli
│  └─ gateway
├─ papers/                        T1-T2 drafts, data and scripts, one folder each
├─ spikes/                        four throwaway experiments, each with a README holding the result
│  ├─ s1-litellm-core/  s2-claude-code-402/  s3-model-switch-cache-loss/  s4-subagent-multiplier/
├─ tools/                         check_layering.py, check_extractable.py, check_surface.py
├─ tests/integration/             end-to-end replay through apps/gateway
├─ docs/adr/                      0001-gateway-core-and-ledger-store.md
└─ reports/baseline/              30-day baseline, checked in
```

Gates

Two kinds. Pull request checks run on every change and are cumulative. The exit gate runs once, and a person other than the builder signs it.

REQUIRED CHECK	WHAT IT PROVES	RUNS
`lint, types`	Ruff and mypy --strict pass per package	Every PR

REQUIRED CHECK	WHAT IT PROVES	RUNS
`unit`	Fast tests, under 2 minutes	Every PR
`contract`	Every harness adapter passes the same suite for the five functions	Every PR
`layering`	import-linter: no package imports a sibling or a higher layer	Every PR
`extractable`	Each touched package and its dependents build and pass tests in a clean venv with only declared deps	Every PR
`replay`	Recorded traces return byte-identical responses through the gateway	Every PR
`ledger-invariants`	One row per response id; buckets are exclusive; unknown usage key is rejected; price book is append-only	Every PR
`secret-scan, pr-title`	No keys in the diff; the title follows Conventional Commits	Every PR
`reconcile-vendor`	Computed cost against the vendor report, within 1 percent	Nightly, not a PR check

Exit gate. The milestone closes when every line is true:

- Computed cost is within 1 percent of the vendor report for two weeks on each provider in use.
- Every request has a cost_source and a price_row_id.
- A 30-day baseline exists: mean, median and P95 spend per developer, cache hit per caller, failed-run share.
- The four spike results are written in 09-risks-and-open-questions.md.

TDD format

1. Write the test from the card's "done when" line. The test name states the behaviour.
2. Cover normalisation and cost maths with property tests: the five buckets always sum to the provider total.
3. Run every adapter through one shared contract suite. A new adapter starts by failing it.
4. Record real traces once, scrub them, and replay them. Replay is the main safety net while nothing is enforced.
5. A spike is not TDD. Time-box it to 3 days and end it with a written result and an ADR if it changes a plan.

```
def test_duplicate_stream_writes_one_ledger_row():
    # Given a recorded stream the provider delivered twice
    trace = load_trace("fixtures/traces/claude_code_dup_stream.jsonl")
    # When both copies pass through the gateway
    replay(trace, gateway)
    # Then the ledger holds one row for that response id
    assert ledger.count(response_id=trace.response_id) == 1
```

Cards

49 cards under 7 epics. Create each epic as an issue and add the cards as its sub-issues. Labels: `phase:P0` plus `type:*` and `area:*`.

EPIC	TYPE	CARD TITLE
Repo setup	CHORE	`chore(repo): scaffold folders, CI, issue forms, PR template and CODEOWNERS`
	CHORE	`chore(repo): add rulesets, required checks and the ci-gate job`

EPIC	TYPE	CARD TITLE
	CHORE	chore(repo): uv workspace with one pyproject per package
	CHORE	chore(repo): import-linter layering contract generated from layout.json
	TEST	test(repo): extraction test builds each package in a clean venv with only its declared deps
	DOCS	docs(adr): 0001 gateway core and ledger store
Gateway in observe mode	FEAT	feat(gateway): proxy requests with virtual keys
	FEAT	feat(gateway): tee streamed responses without delaying the client
	FEAT	feat(adapters): define route, identify, observe, cap and replay
	FEAT	feat(adapters): Claude Code adapter through the base URL
	FEAT	feat(adapters): SDK adapter
	FEAT	feat(adapters): Codex CLI adapter
	TEST	test(gateway): replayed trace returns byte-identical responses
Ledger and price book	FEAT	feat(usage): normalise usage into five exclusive buckets
	FEAT	feat(usage): reject unknown usage keys
	FEAT	feat(ledger): one row per provider response id
	FEAT	feat(pricing): append-only price book with a 30-day staleness flag
	FEAT	feat(pricing): cost calculator in the order of 04-cost-formula
	FEAT	feat(ledger): attribution by tenant, team, agent, session and prompt
	FEAT	feat(reconcile): daily comparison with the vendor report
Detectors in alert mode	FEAT	feat(detectors): detector base class and fired-count table
	FEAT	feat(detectors): row 7 alert, interrupted turns
	FEAT	feat(detectors): row 9 alert, no clear direction
	FEAT	feat(detectors): row 10 alert, restarting too often
	FEAT	feat(detectors): row 22 alert, correction churn
	FEAT	feat(detectors): row 26 alert, repeated tasks across sessions
	FEAT	feat(detectors): row 35 alert, bloated json and query results
	FEAT	feat(detectors): row 38 alert, copied history
	FEAT	feat(detectors): row 41 alert, dependency, vendor and generated files
	FEAT	feat(detectors): row 44 alert, re-learning the codebase
	FEAT	feat(detectors): row 46 alert, tool discovery and skill loading
	FEAT	feat(detectors): row 47 alert, usage while idle
	FEAT	feat(detectors): row 48 alert, mcp tool schema bloat
	FEAT	feat(detectors): row 49 alert, unused skills/plugins
	FEAT	feat(detectors): row 50 alert, no visibility
	FEAT	feat(detectors): alert-mode detectors for rows 1, 2, 3, 4, 5, 6, 8, 11, 12

EPIC	TYPE	CARD TITLE
	FEAT	feat(detectors): alert-mode detectors for rows 13, 14, 15, 16, 17, 18, 19, 20, 21
	FEAT	feat(detectors): alert-mode detectors for rows 23, 24, 25, 27, 28, 29, 30, 31, 32
	FEAT	feat(detectors): alert-mode detectors for rows 33, 34, 36, 37, 39, 40, 42, 43, 45
	DOCS	docs(detectors): coverage table, 50 rows with fired counts over the baseline
Spikes	SPIKE	spike(gateway): LiteLLM as the gateway core
	SPIKE	spike(claude-code): behaviour when the gateway returns a 402
	SPIKE	spike(cache): cache loss on a model switch
	SPIKE	spike(agents): subagent token multiplier
	DOCS	docs(risks): write the four spike results into 09-risks-and-open-questions
Baseline and gate	FEAT	feat(report): 30-day baseline of spend per developer, cache hit and failed-run share
Research papers	DOCS	docs(paper): T1 Cost per Successful Task, Specified: Numerator, Denominator and Failure Handling for Billed Coding-Agent Spend
	DOCS	docs(paper): T2 Five Exclusive Token Buckets: Auditing Usage Schemas Across Five Vendors
Baseline and gate	GATE	gate(P0): exit review, cost within 1 percent for two weeks per provider

P1 · 4 WEEKS · MILESTONE P1 ENFORCE, TAG V0.2.0

P1: *enforce*

Move limits outside the agent. Budgets, caps, loop guard and fallbacks.

6	20	6	4
epics	cards	PR and nightly checks	exit-gate lines

Folder structure

Packages this phase adds, plus the shared folders it touches. Every package has its own pyproject, tests, intro file (README) and changelog. Layering rules are in the repository section.

```
token-economics/                          (earlier packages stay; new ones listed)
├─ features/  (5 new)
│  ├─ budgets
│  ├─ headless-runner
│  ├─ loop-guard
│  ├─ max-tokens
│  └─ turn-cap
├─ services/  (5 new)
│  ├─ health
│  ├─ insight
│  ├─ policy
│  ├─ reconciler
│  └─ tenancy
├─ apps/  (2 new)
│  ├─ admin-api
│  └─ worker
├─ papers/                          T3-T5 drafts, data and scripts
├─ tests/
│  ├─ fault/                        provider 429, timeout, stream cut
│  ├─ concurrency/                  budget overshoot under parallel load
│  └─ capability/                   capability matrix, baseUrl false never enforced
└─ docs/adr/                        0002-fail-open-observe-fail-closed-budget.md
```

Gates

Two kinds. Pull request checks run on every change and are cumulative. The exit gate runs once, and a person other than the builder signs it.

REQUIRED CHECK	WHAT IT PROVES	RUNS
`fault`	Injected 429, timeout and stream cut never double-count or lose a ledger row	Every PR
`budget-overshoot`	Parallel requests stay within the budget plus the measured overshoot	Every PR
`failure-posture`	Gateway down: observation fails open, budgets fail closed	Every PR

REQUIRED CHECK	WHAT IT PROVES	RUNS
`capability-matrix`	No harness with baseUrl false is marked enforced	Every PR
`guard-replay`	Each class 1 guard leaves task success unchanged on the replay set	Every PR touching guards/
all P0 checks	Still required	Every PR

Exit gate. The milestone closes when every line is true:

- A test key cannot exceed its budget by more than the measured overshoot.
- Fault injection never double-counts or loses a ledger row.
- Fail-open for observation and fail-closed for budgets are both demonstrated.
- No harness with baseUrl = false is marked as enforced.

TDD format

1. Write the failure first. Each enforcement card starts with the test that shows the limit being broken.
2. Test enforcement with fault injection, not with mocks of the happy path.
3. Measure the overshoot once, record the number in the test, and fail the build if it grows.
4. Every guard ships with a replay test that compares task success before and after.
5. Turn caps use the P75 value from the P0 baseline file, not a constant in code.

```
def test_key_never_exceeds_budget_by_more_than_measured_overshoot():
    # Given a key with a 10 dollar budget and the overshoot bound from the P0 baseline
    key = make_key(budget_usd=10, overshoot_bound=OVERSHOOT_BOUND)
    # When 50 parallel requests arrive
    run_parallel(gateway, key, n=50)
    # Then total billed cost stays within budget plus the bound
    assert billed_usd(key) ⩽ 10 + OVERSHOOT_BOUND
```

Cards

20 cards under 6 epics. Create each epic as an issue and add the cards as its sub-issues. Labels: `phase:P1` plus `type:*` and `area:*`.

EPIC	TYPE	CARD TITLE
Budgets	FEAT	feat(budget): per-key budget with reserve then true-up
	FEAT	feat(budget): return 402 or 429 at the gateway when a key is over budget
	FEAT	feat(budget): alerts at 75, 90 and 100 percent
	TEST	test(budget): overshoot stays inside the measured bound
Run control	FEAT	feat(policy): turn cap at P75 of each model baseline, with a remaining-turns reminder
	FEAT	feat(policy): loop guard stops at the first repeated-failure signature
Reliability	FEAT	feat(health): backend health and rate-limit tracking
	FEAT	feat(health): fallback route when a backend fails
	TEST	test(fault): provider 429, timeout and stream cut never double-count or lose a row
	TEST	test(posture): fail-open for observation, fail-closed for budgets

EPIC	TYPE	CARD TITLE
	FEAT	feat(adapters): capability flags, no harness with baseUrl false marked enforced
Zero-risk guards	FIX	fix(row-15): agent startup cost, replay shows success unchanged
	FIX	fix(row-36): wordy and repeated output, replay shows success unchanged
	FIX	fix(row-42): verbose agent reports and briefs, replay shows success unchanged
Reconcile and dashboard	FEAT	feat(reconcile): invoice comparison and weekly canary
	FEAT	feat(dashboard): spend by tenant, caller and model with median and P95
Research papers	DOCS	docs(paper): T3 Where the Billed Dollar Goes: A Four-Component Cost Decomposition of Three Production Coding Harnesses
	DOCS	docs(paper): T4 How Much Does a Soft Budget Leak? Overshoot of Post-Hoc vs Reserve-and-True-Up Caps
	DOCS	docs(paper): T5 Telling the Agent vs Stopping the Agent: Visible Budget Reminders vs Gateway Caps
Reconcile and dashboard	GATE	gate(P1): exit review, key cannot exceed budget beyond the measured overshoot

P2 · 6 WEEKS · MILESTONE P2 OPTIMISE, TAG V0.3.0

P2: *optimise*

Ship each optimisation behind its own flag, only after a paired win in cost per successful task (CPS).

8	59	7	3
epics	cards	PR and nightly checks	exit-gate lines

Folder structure

Packages this phase adds, plus the shared folders it touches. Every package has its own pyproject, tests, intro file (README) and changelog. Layering rules are in the repository section.

```
token-economics/                          (earlier packages stay; new ones listed)
├─ modules/  (1 new)
│  └─ stats
├─ features/  (15 new)
│  ├─ cache-hit-monitor
│  ├─ cache-planner
│  ├─ cascade
│  ├─ compaction-guard
│  ├─ context-clearing
│  ├─ instruction-lint
│  ├─ observation-masking
│  ├─ output-cap
│  ├─ probe
│  ├─ prompt-lint
│  ├─ rules-router
│  ├─ schema-deferral
│  ├─ static-router
│  ├─ sticky-model
│  └─ switch-pricer
├─ services/  (2 new)
│  ├─ eval
│  └─ rollout
├─ papers/                          T6-T9 drafts, data and scripts
├─ evals/
│  ├─ tasks/private/                hard cases, kept out of public CI logs
│  ├─ tasks/lite80/                 80-task subset
│  ├─ bounds/                       non-inferiority bound per test, committed before the run
│  └─ results/row-NN/               <harness>-<version>.json, one per row, win or lose
├─ docs/evals/OFF.md                fixes that failed, with the reason
└─ docs/adr/                        0003-router-only-if-it-beats-best-single.md
```

Gates

Two kinds. Pull request checks run on every change and are cumulative. The exit gate runs once, and a person other than the builder signs it.

REQUIRED CHECK	WHAT IT PROVES	RUNS
`eval-dry-run`	A no-op change reports no difference	Every PR touching evals/
`bound-committed`	The non-inferiority bound file is older than the result file in git history	Every fix PR
`result-file-present`	Fix PR includes evals/results/row-NN/ or an entry in OFF.md	Every fix PR
`flag-default-off`	Every new flag defaults to off	Every PR
`bootstrap-power`	The paired bootstrap finds a known synthetic effect and misses none for a no-op	Every PR touching evals/
all P0 and P1 checks	Still required	Every PR
`eval-run`	Full paired run on the private set	Manual or label needs-eval, not a PR check

Exit gate. The milestone closes when every line is true:

- Every shipped optimisation has a paired result showing CPS down, with the success bound held.
- Any optimisation that fails stays off. That counts as a pass.
- The router beats the best single model after its own overhead and cache loss, or it is not enabled.

TDD format

1. Declare the hypothesis and the non-inferiority bound in the card before writing code. Commit the bound file first.
2. Red means the eval shows no win. Green means a paired win with the success bound held.
3. A fix that never reaches green is merged as "off" with a result file. Do not delete it.
4. Test the harness before the fixes: a no-op change must report no difference.
5. Each arm gets its own cache. Otherwise the second arm reads the first arm's cache and the win is fake.

```
def test_paired_bootstrap_reports_no_difference_for_a_noop_change():
    # Given the same gateway config on both arms
    result = paired_bootstrap(arm_a=BASELINE, arm_b=BASELINE, tasks=LITE80, seed=7)
    # Then the CPS interval contains zero and the success bound holds
    assert result.cps_ci.contains(0.0)
    assert result.success_within_bound
```

Cards

59 cards under 8 epics. Create each epic as an issue and add the cards as its sub-issues. Labels: `phase:P2` plus `type:*` and `area:*`.

EPIC	TYPE	CARD TITLE
Eval harness	FEAT	feat(eval): private task set with hard cases
	FEAT	feat(eval): 80-task Lite subset
	FEAT	feat(eval): paired task-clustered bootstrap

EPIC	TYPE	CARD TITLE
	FEAT	feat(eval): non-inferiority bound declared before each test
	FEAT	feat(eval): isolated cache per arm
	TEST	test(eval): a no-op change reports no difference
Cache planner	FEAT	feat(cache): TTL from the observed reuse gap
	FEAT	feat(cache): per-caller hit-rate monitor
Context guard	FEAT	feat(context): schema deferral
	FEAT	feat(context): output caps
	FEAT	feat(context): observation masking
	FEAT	feat(context): tool-result clearing
	FEAT	feat(context): compaction, last and only if the others fall short
Router	FEAT	feat(router): rules router
	FEAT	feat(router): atomic classifier
	FEAT	feat(router): best-single-model baseline
	FEAT	feat(router): static router on execution-verified labels, only if it beats the baseline
	FEAT	feat(router): cascade, only if measured acceptance beats the cost ratio
Shadow runs	FEAT	feat(shadow): mirrored traffic with write isolation
	DOCS	docs(shadow): shadow report with unplanned side effects
Behaviour-change fixes	FIX	fix(row-1): heavy starting context, paired CPS test and result file
	FIX	fix(row-2): cache-write premium, paired CPS test and result file
	FIX	fix(row-3): cold cache at session and agent start, paired CPS test and result file
	FIX	fix(row-4): cache expired after a break, paired CPS test and result file
	FIX	fix(row-5): many agents, paired CPS test and result file
	FIX	fix(row-6): long history replay, paired CPS test and result file
	FIX	fix(row-8): sessions that never end, paired CPS test and result file
	FIX	fix(row-11): stale tool results carried forward, paired CPS test and result file
	FIX	fix(row-12): bigger model than needed, paired CPS test and result file
	FIX	fix(row-13): fork context duplication, paired CPS test and result file
	FIX	fix(row-14): large tool-call arguments, paired CPS test and result file
	FIX	fix(row-16): model switch rewrote the cache, paired CPS test and result file
	FIX	fix(row-17): old replies replayed, paired CPS test and result file
	FIX	fix(row-18): clarification loops, paired CPS test and result file
	FIX	fix(row-19): screenshot accumulation, paired CPS test and result file
	FIX	fix(row-20): visual iteration loops, paired CPS test and result file
	FIX	fix(row-21): cache not reused, paired CPS test and result file
	FIX	fix(row-23): agents racing the cache, paired CPS test and result file

EPIC	TYPE	CARD TITLE
	FIX	fix(row-24): too much tool output, paired CPS test and result file
	FIX	fix(row-25): noisy and repeated terminal output, paired CPS test and result file
	FIX	fix(row-27): failed, retried and polling calls, paired CPS test and result file
	FIX	fix(row-28): repeated screenshots, paired CPS test and result file
	FIX	fix(row-29): progress narration, paired CPS test and result file
	FIX	fix(row-30): whole-file reads, paired CPS test and result file
	FIX	fix(row-31): duplicate reading across agents, paired CPS test and result file
	FIX	fix(row-32): agent coordination chatter, paired CPS test and result file
	FIX	fix(row-33): large images and pdfs, paired CPS test and result file
	FIX	fix(row-34): broad or unbounded searches, paired CPS test and result file
	FIX	fix(row-37): web pages and search results, paired CPS test and result file
	FIX	fix(row-39): compaction cost, paired CPS test and result file
	FIX	fix(row-40): git and github output, paired CPS test and result file
	FIX	fix(row-43): repeated reads of unchanged files, paired CPS test and result file
	FIX	fix(row-45): high effort on trivial turns, paired CPS test and result file
	DOCS	docs(evals): the off list with a reason for each failed row
Research papers	DOCS	docs(paper): T6 Paired, Task-Clustered Evaluation of Cost Optimisations with Cache Isolation and Non-Inferiority Margins
	DOCS	docs(paper): T7 5 Minutes or 1 Hour? Planning Prompt-Cache Lifetime from Measured Reuse-Gap Distributions
	DOCS	docs(paper): T8 Does a Router Beat the Best Single Model After Overhead and Cache Loss?
	DOCS	docs(paper): T9 Same Compressor, Same Model, Different Harness: Why Output Compression Moves Billed Cost in Opposite Directions
Gate	GATE	gate(P2): exit review, one result file per candidate row

P3 · 4 WEEKS · MILESTONE P3 HARDEN, TAG V1.0.0

P3: *production hardening*

Make every shipped fix reversible, secure and owned.

7	26	6	4
epics	cards	PR and nightly checks	exit-gate lines

Folder structure

Packages this phase adds, plus the shared folders it touches. Every package has its own pyproject, tests, intro file (README) and changelog. Layering rules are in the repository section.

```
token-economics/                          (earlier packages stay; new ones listed)
├─ features/  (3 new)
│  ├─ anomaly
│  ├─ strong-share
│  └─ tier-selector
├─ papers/                          T10-T13 drafts, T14 outline
├─ tests/
│  ├─ chaos/                        rollback drills, one per trigger
│  └─ security/                     storage scan for prompt content
├─ ops/
│  ├─ alerts.yml                    every alert has an owner field
│  ├─ loadtest/                     scripts and the latency budget from P0
│  └─ dr/                           disaster recovery drill notes
├─ docs/runbooks/                    one file per alert
└─ SECURITY.md
```

Gates

Two kinds. Pull request checks run on every change and are cumulative. The exit gate runs once, and a person other than the builder signs it.

REQUIRED CHECK	WHAT IT PROVES	RUNS
`kill-switch-bytes`	With a switch off, responses match the P1 baseline byte for byte	Every PR
`storage-scan`	No prompt content in the store, logs or traces by default	Every PR
`alert-owner-lint`	Every alert in ops/alerts.yml has an owner and a runbook file	Every PR
`rollback-drill`	Each trigger reverts a bad fix without a deploy	Nightly
`load-test`	Gateway holds the P0 latency budget	Nightly and before each release
all earlier checks	Still required	Every PR

Exit gate. The milestone closes when every line is true:

- A rollback drill reverts a bad optimisation without a deploy.
- A load test holds the gateway latency budget set in P0.
- Security review is closed. No prompt content is stored by default.
- A named owner exists for each alert.

TDD format

1. Write the drill first. Each rollback card starts as a chaos test that injects the bad condition.
2. Test every kill-switch in the off position: the response must match the earlier baseline.
3. Run the storage scan as a test. It fails if a prompt string from a fixture appears anywhere on disk.
4. Treat runbooks as tested code: a drill that cannot follow the runbook fails the card.

```
def test_rollback_fires_when_success_falls_below_floor_without_deploy():
    # Given a shipped fix whose success rate drops below the floor
    fix = enable("row_11_clearing", tenant="acme")
    inject_success_drop(fix, below_floor=True)
    # When the rollback trigger evaluates
    run_triggers()
    # Then the fix is off for that tenant and no deploy happened
    assert not flags.is_on("row_11_clearing", tenant="acme")
    assert deploys.count() == 0
```

Cards

26 cards under 7 epics. Create each epic as an issue and add the cards as its sub-issues. Labels: `phase:P3` plus `type:*` and `area:*`.

EPIC	TYPE	CARD TITLE
Canary and kill-switch	FEAT	feat(rollout): canary stage for every shipped fix
	FEAT	feat(rollout): kill-switch per optimisation and tenant
Auto-rollback	FEAT	feat(rollback): trigger when success falls below the floor
	FEAT	feat(rollback): trigger when cost per success exceeds the baseline interval
	FEAT	feat(rollback): trigger when cache hit rate drops
	FEAT	feat(rollback): trigger when billed cost diverges from the estimate
	TEST	test(rollback): drill for each trigger, no deploy
Security	FEAT	feat(security): secrets handling
	FEAT	feat(security): authorisation per tenant and role
	FEAT	feat(security): retention and redaction, no prompt content stored by default
	TEST	test(security): storage scan finds no prompt content
Multi-tenant	FEAT	feat(tenancy): per-tenant limits
	FEAT	feat(tenancy): strong-share ceiling per tenant
Audit and admin	FEAT	feat(audit): append-only audit log

EPIC	TYPE	CARD TITLE
	FEAT	feat(admin): admin API for keys, budgets and switches
Operations	DOCS	docs(runbooks): one runbook per alert
	CHORE	chore(ops): a named owner for each alert
	TEST	test(load): gateway holds the latency budget set in P0
	TEST	test(dr): disaster recovery drill
Research papers	DOCS	docs(paper): T10 When Does Another Attempt Stop Paying? Dollar-Denominated Stopping Rules for Agent Retries and Step Caps
	DOCS	docs(paper): T11 Bounding the Bill, Not Detecting the Prompt: Strong-Model Share Ceilings as a Router Defence
	DOCS	docs(paper): T12 What a Gateway Can See That a Transcript Cannot, and the Reverse: Waste Detection Across Two Vantage Points
	DOCS	docs(paper): T13 Price Books Go Stale: Lifecycle and Canaries for Billing-Accurate Cost Tracking
	DOCS	docs(paper): T14 system paper outline, data freeze at v1.0.0
Security	GATE	gate(security): review closed with sign-off
Operations	GATE	gate(P3): exit review and v1.0.0 release

SOURCES

Where the content *comes from*

- Phases, exit gates and done-when tables: `plan/PHASES.md`.
- Row numbers, names and classes (0 alert, 1 guard, 2 behaviour change, 3 human action): `plan/12-fix-table-50-rows.md`. The split is 14 rows in P0, 3 in P1 and 33 in P2.
- Monorepo layout and package map: `plan/13-monorepo-layout.md`. Paper titles and calendar: `plan/14-research-papers.md`.
- Repository mechanics, check names, board columns, work-in-progress limit, TDD cycle, card titles and version tags: DevX proposal.
- Durations assume three engineers and are estimates.

REFERENCES

Research papers *to read*

All 30 papers cited in this update, with links to the arXiv page. The first 19 were read in full text. The last group was cited from abstracts and a search, so check them before quoting. Dates are the arXiv v1 submission date, except the earlier-context rows, which show the month from the identifier.

Routing (read in full)

PAPER	DATE	WHAT THE FULL TEXT SHOWED
AgentRouter: Heterogeneous Model Routing for Cost-Optimal Multi-Step Agentic Workflows (2609.22951)	2026-09-19	71.8% cost cut at 97.3% quality, but against GPT-4o on four templated domains.

PAPER	DATE	WHAT THE FULL TEXT SHOWED
COMPASS: Routing via Capability-Oriented Model Profiling with Adaptive Search, Selection, and Tool Use (2610.27822)	2026-10-20	78.1% at $215 against 74.1% at $500, excluding router inference and needing a 28-configuration pool.
Pathwise-Budgeted LLM Routing: Online Allocation with K-Bit Tail-Cost Learning (2611.05835)	2026-11-04	Over 2x tasks solved only at the tightest budget.
Chance-Constrained Personalized LLM Routing with Online Preference Learning (2610.22695)	2026-10-15	Asymptotic guarantee, simulated satisfaction, no savings shown.
Rethinking Quality in LLM Routing: Security-Conditioned Model-and-Budget Selection (SCRouter) (2610.18566)	2026-10-13	Quality-only routing can pick insecure pairs. Gain shrinks out of distribution.
Which Model for Which Role? Cost-Efficient Pareto Recommendation for LLM Agentic Workflows (2610.26676)	2026-10-19	A substantial fraction of the frontier at about 10% of the search cost, most by 30%.
Ensemble Routing for Prompt-Adaptive Cost-Aware LLM Inference (PISA) (2611.10193)	2026-11-06	Ensemble gains are marginal. A fixed single model often matches them.
MultiTurnRoute: A Platform for Building and Evaluating Multi-Turn LLM Routers (2611.10848)	2026-11-07	20.3% is a simulated saving on chat traces.

Context control and measurement (read in full)

PAPER	DATE	WHAT THE FULL TEXT SHOWED
TokenEconomizer: Experience-Driven Black-Box Cost Control for Coding Agents (2610.22732)	2026-10-15	18.4 to 30.9% cache-aware cost cut on SWE-Bench. Harness-side, three mechanisms.
CliffCompaction: Cost-Efficient Compaction for Long-Horizon Coding Agents (2609.26779)	2026-09-22	Cache reads are 78% of the bill in one setting. Real Kimi savings are about 0%.
An Empirical Study of Harness Design for Coding Agents (2609.20804)	2026-09-17	Elision and summarisation reach similar accuracy. A staged mix is cheapest.
Beyond Token Savings: A Systematic Study of Context Compression in LLM Agents (2609.32961)	2026-09-26	Fewer tokens did not reliably mean cheaper or faster. Costs are input-only estimates.
Total Cost of Agency: Exact Attribution of Memory Injection Cost in Multi-Agent Workflows (2609.23790)	2026-09-20	13.6% injection share in a synthetic enterprise workflow.
Identical Runs, Different Results: Benchmarking AI Coding Agents on Open-Weight Models (2609.33812)	2026-09-27	584 runs over three studies. 22.5x cost gap from a rate card.
DeltaSelect: Affordable A/B Testing for Coding Agents (2609.19607)	2026-09-17	13 evaluations for $27.86 on 8 tasks. Score gain not significant.

Latent and key-value cache compression (read in full, open-weight models only)

PAPER	DATE	WHAT THE FULL TEXT SHOWED
Tools Need Not Speak in Text: Latent Context Compression for Agents (Coca-Cola) (2611.06500)	2026-11-04	70.01 to 89.24% fewer tokens on web-search question answering. One compressor per agent.
In-Model Context Compression with Agentic Soft Tokens for Long-Horizon LLM Agents (ASTC) (2610.41291)	2026-10-30	50.68% fewer attention pairs in the best case, 19.30% on AppWorld.
Compress What You See, Not What You Say: Anchored Context Distillation for Latent-Observation Software Engineering Agents (LOHA/ACD) (2609.31430)	2026-09-25	43 to 57% less context per call, about 8% over a whole trajectory.
ActKV: Efficient LLM Agents through Action-Guided KV Cache Management (2609.31395)	2026-09-25	98.53% accuracy at 25.98% peak KV memory, offline on one GPU.

Earlier context (cited from abstracts, not read in full)

PAPER	DATE	WHAT THE FULL TEXT SHOWED
CoACT (2607.02911)	2026-07	Text-level compression that preserves the next action.
TokenPilot (2606.17016)	2026-06	Cache-stable compaction.
Token Reduction Is Not Cost Reduction (2607.12161)	2026-07	Token cuts that did not lower the bill.
Prompt-Induced Waste in Coding Agents (2608.01347)	2026-08	Prompt wording that adds waste.
What Does Context Compression Cost an Agent? (2608.16370)	2026-08	Dropped state is re-fetched.
LENS (2608.16185)	2026-08	Cited in the latent-compression search.
Not Worth Another Token (2608.08389)	2026-08	Cited in the latent-compression search.
Control Under Compression (2608.01056)	2026-08	Cited in the latent-compression search.
KV-cache scheduling bound (online scheduling, Θ(n) competitive ratio) (2608.16944)	2026-08-14	Only formal bound found, dated before the window.
LatentPress (2609.01507)	2026-09-01	Latent compression, dated on the window's first day.
An Empirical Cost Attribution of Context-Compression Gateways (2609.22114)	2026-08-19	Submitted in August despite the September ID.